Отчёт по лабораторной работе № 3

Измерение и тестирование пропускной способности сети. Воспроизводимый эксперимент

Дисциплина: Моделирование сетей передачи данных
Автор: Авдадаев Джамал Геланиевич
Группа: НФИбд-02-23
Дата выполнения: 10 октября 2026 г.

Содержание

Содержание ..... **Error! Bookmark not defined.**
1. Введение 2
1.1. Цель и задачи 2
1.2. Объект, предмет и термины 2
1.3. Теория и инструменты 2
2. Процесс выполнения 2
2.1. Подготовка и исходная топология 2
2.2. Получение IP- и MAC-адресов 4
2.3. Моделирование ограниченного канала 7
2.4. Сравнение качества передачи ICMP 10
2.5. Подготовка воспроизводимого эксперимента 11
2.6. Автоматический запуск и обработка JSON 14
2.7. Визуализация результатов 16
2.8. GNU Make и проверка воспроизводимости 19
3. Полученные результаты и анализ 21
4. Заключение 21
5. Библиографическая информация 22
6. Перечень иллюстраций и таблиц 22

1. Введение

В лабораторной работе исследована работа небольшой виртуальной сети в Mininet и выполнены измерения с использованием iPerf3. Практическая часть последовательно охватывает исходную топологию, моделирование ограничений канала, автоматический TCP-тест, построение графиков и повторный запуск средствами GNU Make.

1.1. Цель и задачи

Цель: освоить Python API Mininet и организовать воспроизводимый эксперимент по измерению пропускной способности сети.

Задачи: создать топологию h1–s3–h2; проверить сетевую связность и адресацию; задать ограничения CPU и канала; сравнить ICMP-тесты; автоматизировать iPerf3 и экспорт JSON; построить графики; проверить команды make clean и make.

1.2. Объект, предмет и термины

Объект исследования — эмулируемая сеть с двумя хостами и одним коммутатором. **Предмет исследования** — настройки пропускной способности, задержек, потерь и измеренные показатели TCP/ICMP при повторном выполнении эксперимента.

Обозначения: Mininet — эмулятор сетей; iPerf3 — инструмент измерения сетевой производительности; RTT — время прохождения пакета туда и обратно; cwnd — окно перегрузки TCP; HTB — дисциплина обслуживания для ограничения скорости; netem — моделирование задержек и потерь; JSON — структурированный формат результатов.

1.3. Теория и инструменты

Среднеуровневый Python API Mininet позволяет вручную создавать узлы и соединения через addHost(), addSwitch(), addLink(), start() и stop(). TCLink задаёт параметры канала через Linux tc, CPULimitedHost ограничивает процессорное время. TCP-пропускная способность зависит не только от заданного значения bw, но также от RTT, cwnd и условий выполнения эксперимента.

Использованы Windows, VirtualBox, Mininet-VM (Ubuntu 20.04.1 LTS), SSH/PuTTY, Python 3, Mininet, iPerf3, ping, tc, GNU Make, Gnuplot и скрипты iperf3_plotter. В качестве метода проверки применены анализ вывода команд, ICMP-тесты, чтение JSON и визуальный анализ графиков. Для сохранения результатов использованы скриншоты этапов выполнения.

2. Процесс выполнения

2.1. Подготовка и исходная топология

Сначала подготовлен Python-скрипт на основе стандартного примера Mininet. Топология включает хосты h1 и h2, коммутатор s3, контроллер c0. Проверка связности выполнена до внесения ограничений.

В рабочем каталоге создана папка lab_iperf3_topo; исходный пример Mininet скопирован и переименован в lab_iperf3_topo.py.

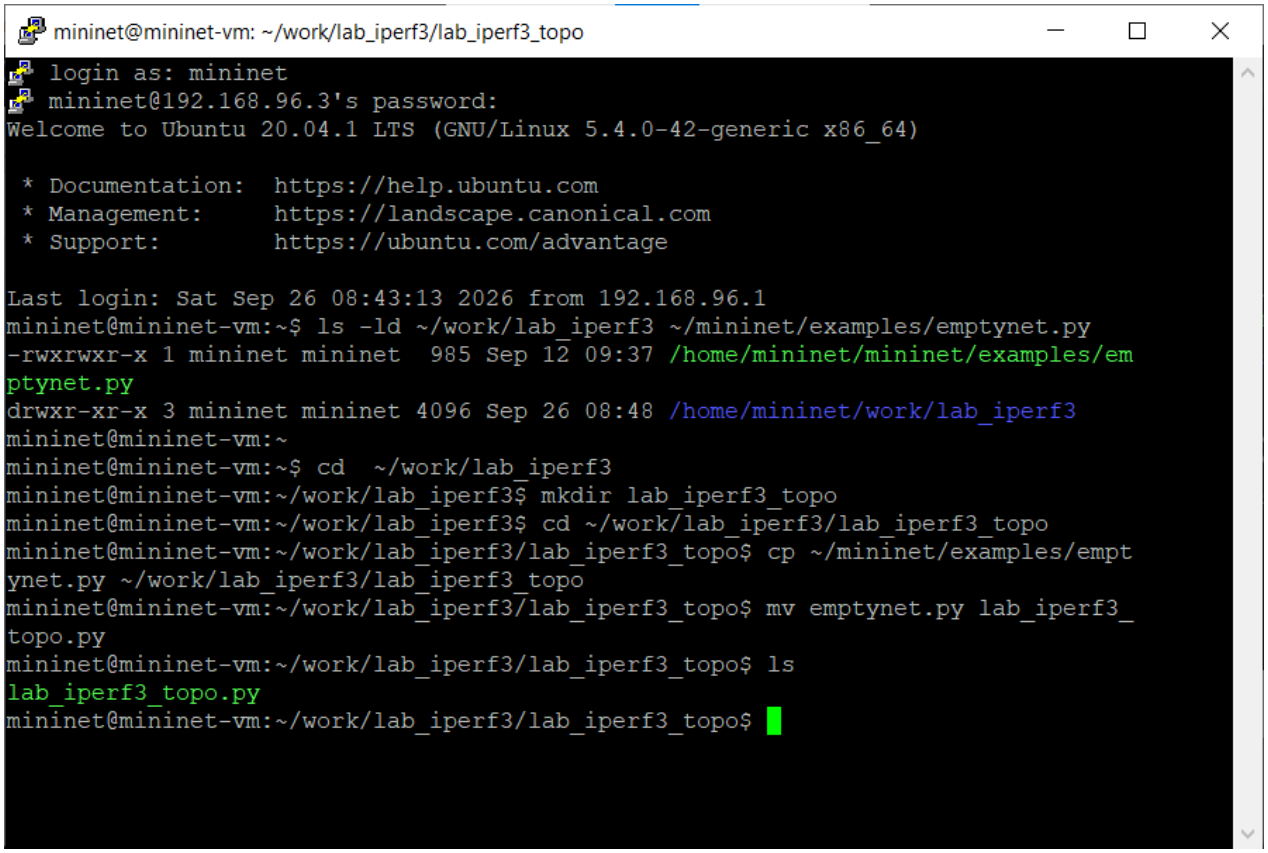


Рис. 1: Подготовка рабочей папки и копирование emptynet.py.

В шаблоне используются методы addController(), addHost(), addSwitch() и addLink(); с их помощью вручную формируется небольшая топология.

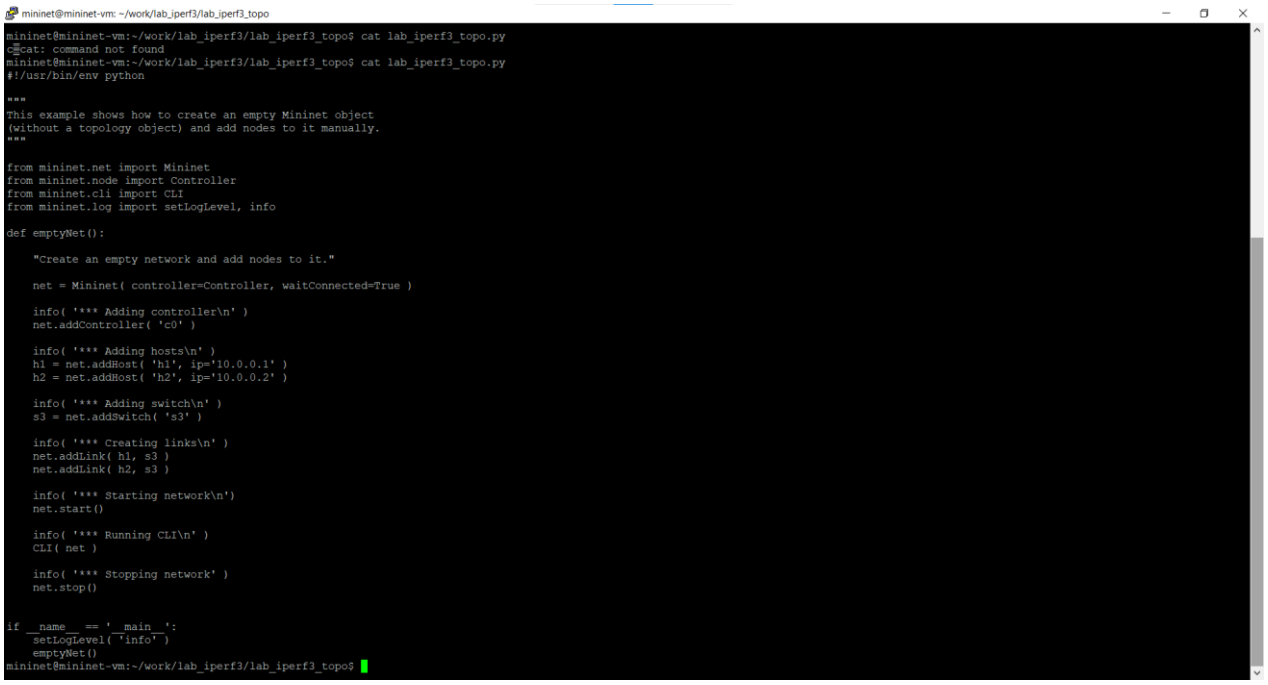


Рис. 2: Исходный скрипт создания сети.

Команда sudo python3 lab_iperf3_topo.py запускает контроллер c0, два хоста, коммутатор s3 и интерактивную оболочку Mininet.

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python3 lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
*** Running CLI
*** Starting CLI:
mininet>
```

Рис. 3: Запуск исходной топологии через Python API.

Команды net, links и dump подтверждают состав топологии; pingall показывает 0% потерь при первичной проверке.

```
mininet> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
s3 lo:  s3-eth1:h1-eth0 s3-eth2:h2-eth0
c0
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=936>
<Host h2: h2-eth0:10.0.0.2 pid=939>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=944>
<Controller c0: 127.0.0.1:6653 pid=929>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Рис. 4: Проверка связности и состава сети.

2.2. Получение IP- и MAC-адресов

После net.start() добавлен автоматический вывод сетевых параметров средствами Python API. Это удобно для проверки того, что созданные узлы получили ожидаемые IP-адреса.

В скрипт включена инструкция, использующая свойства name и методы IP() и MAC() объекта h1.

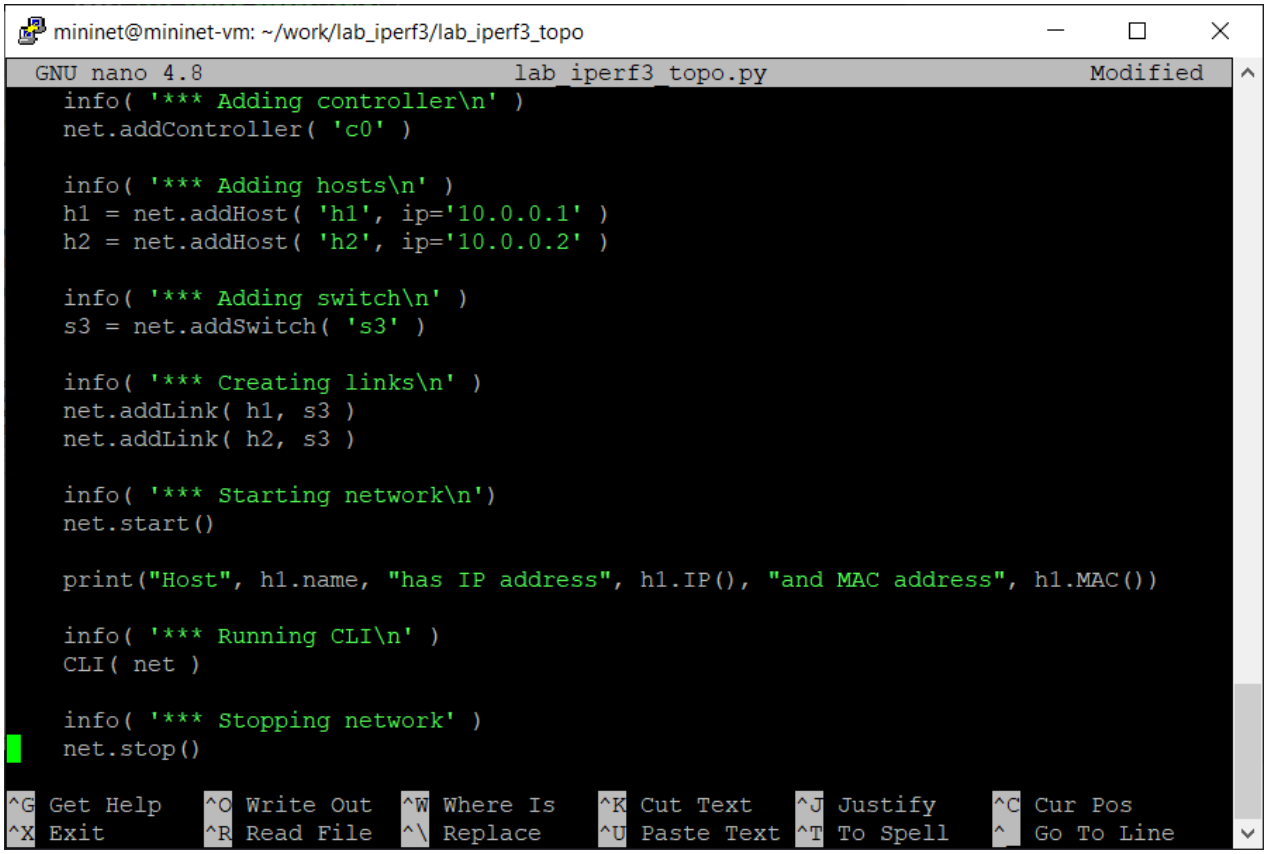


Рис. 5: Добавление вывода параметров хоста h1.

После запуска программа выводит IP-адрес 10.0.0.1 и назначенный хосту MAC-адрес.



Рис. 6: Вывод IP- и MAC-адреса h1.

После строки для h1 добавлен аналогичный вывод имени, IP-адреса и MAC-адреса h2.

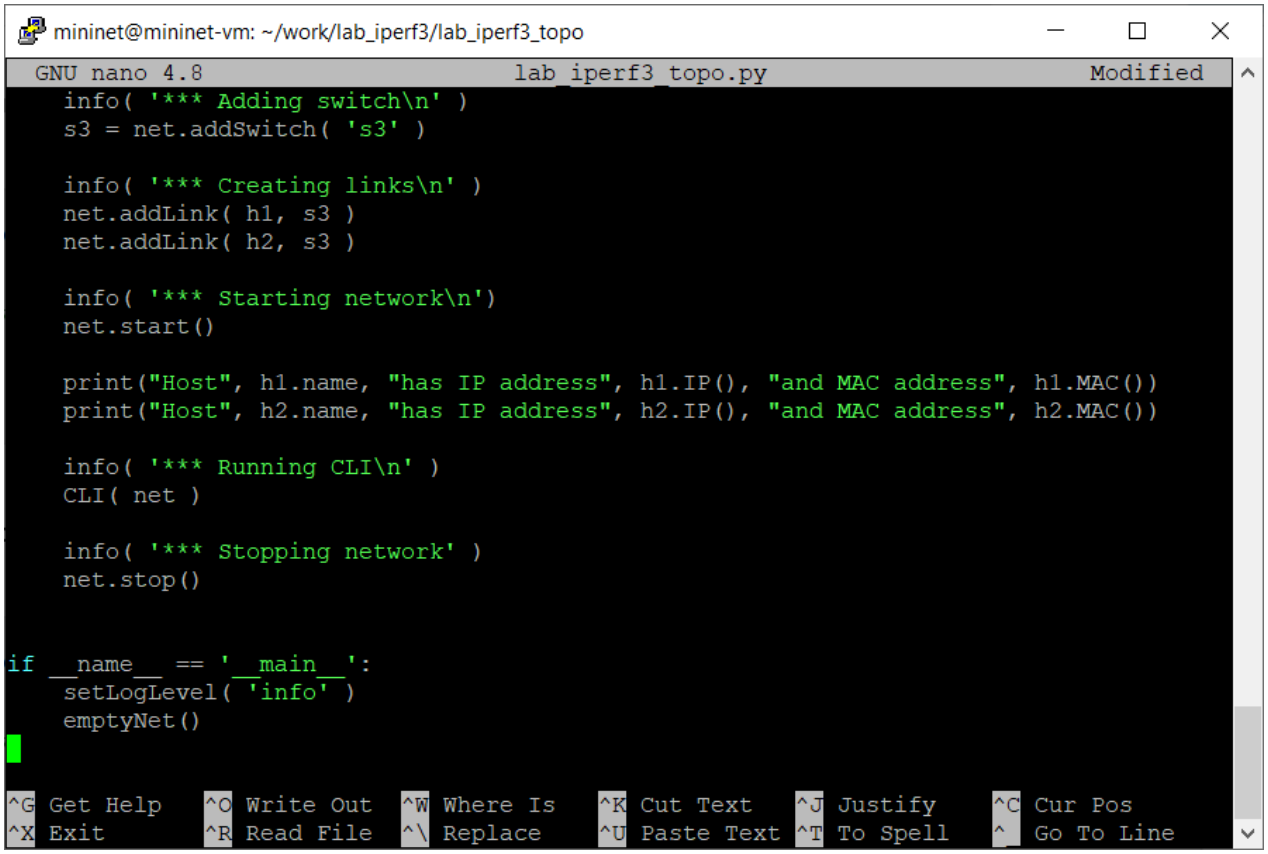


Рис. 7: Добавление вывода параметров второго хоста.

Выполнение кода подтверждает IP-адреса 10.0.0.1 и 10.0.0.2. MAC-адреса могут меняться при повторном создании сети.

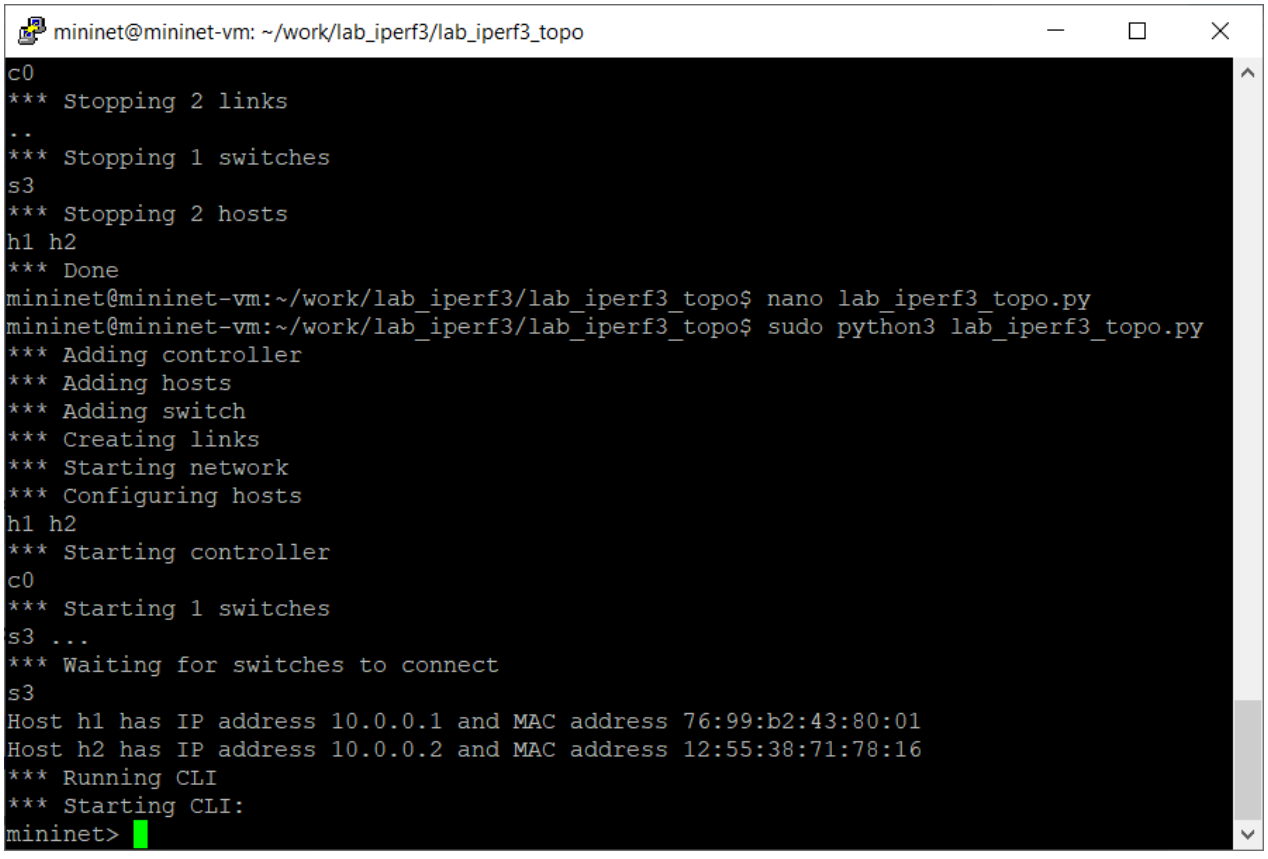


Рис. 8: Результаты получения адресов обоих хостов.

2.3. Моделирование ограниченного канала

Во второй версии скрипта подключены CPULimitedHost и TCLink. Ограничения скорости и потерь применены к линии h1–s3; их действие дополнительно подтверждено средствами Linux tc.

Для моделирования ограниченных ресурсов импортированы CPULimitedHost и TCLink.

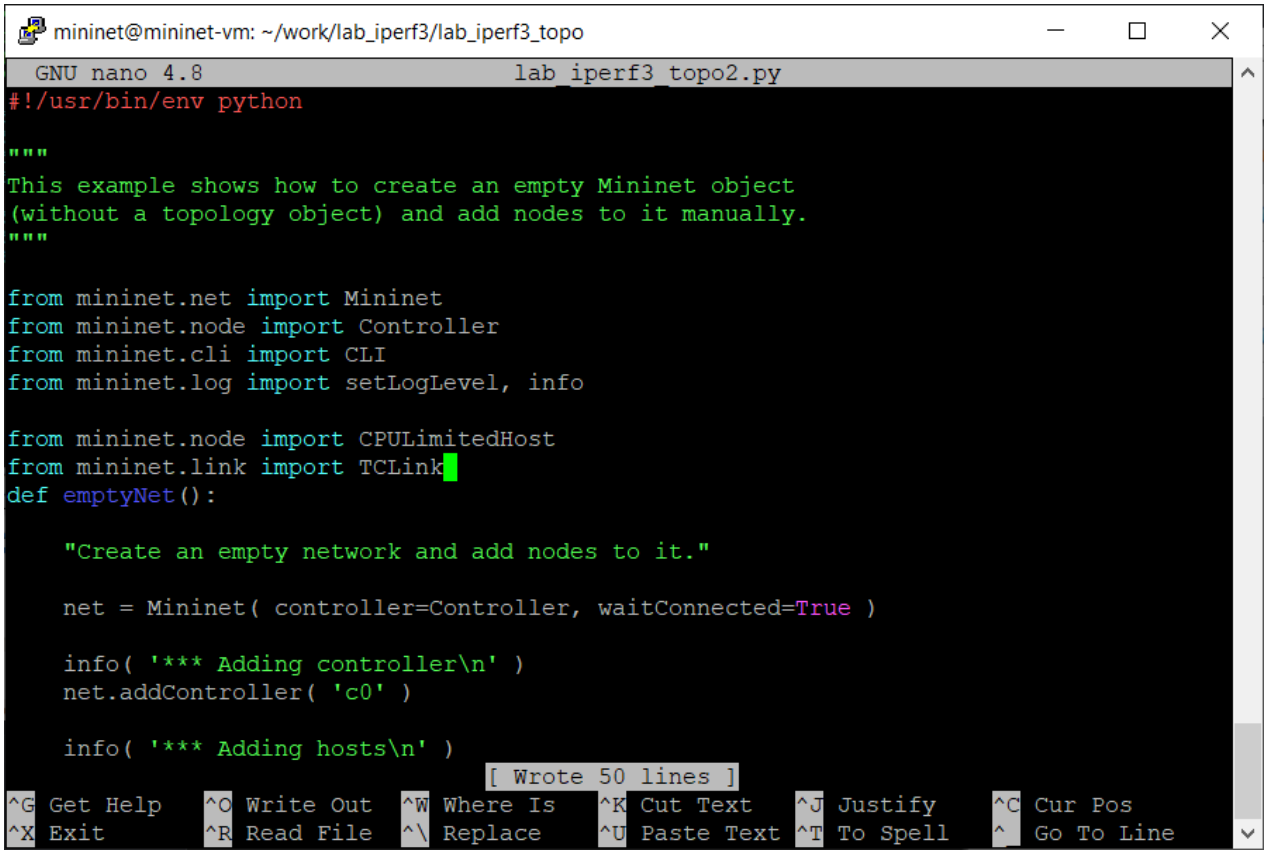


Рис. 9: Подключение классов CPULimitedHost и TCLink.

В конструктор Mininet добавлены параметры host=CPULimitedHost и link=TCLink.

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

```
  GNU nano 4.8                    lab_iperf3_topo2.py                 Modified
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

from mininet.node import CPULimitedHost
from mininet.link import TCLink
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet(
    controller=Controller,
    waitConnected=True,
    host=CPULimitedHost,
    link=TCLink
    )

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify  ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell ^_ Go To Line
```

Рис. 10: Настройка классов узлов и соединений Mininet.

Для h1 указана доля cpu=0.5, для h2 — cpu=0.45; точная квота определяется Mininet с учётом числа доступных CPU.

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

```
  GNU nano 4.8                    lab_iperf3_topo2.py                 Modified
from mininet.cli import CLI
from mininet.log import setLogLevel, info

from mininet.node import CPULimitedHost
from mininet.link import TCLink
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet(
    controller=Controller,
    waitConnected=True,
    host=CPULimitedHost,
    link=TCLink
    )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1', cpu=0.5 )
    h2 = net.addHost( 'h2', ip='10.0.0.2', cpu=0.45 )

    info( '*** Adding switch\n' )

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify  ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell ^_ Go To Line
```

Рис. 11: Назначение ограничений CPU хостам.

Для линии h1–s3 задано bw=10, delay=5ms, loss=10, max_queue_size=1000 и use_htb=True.

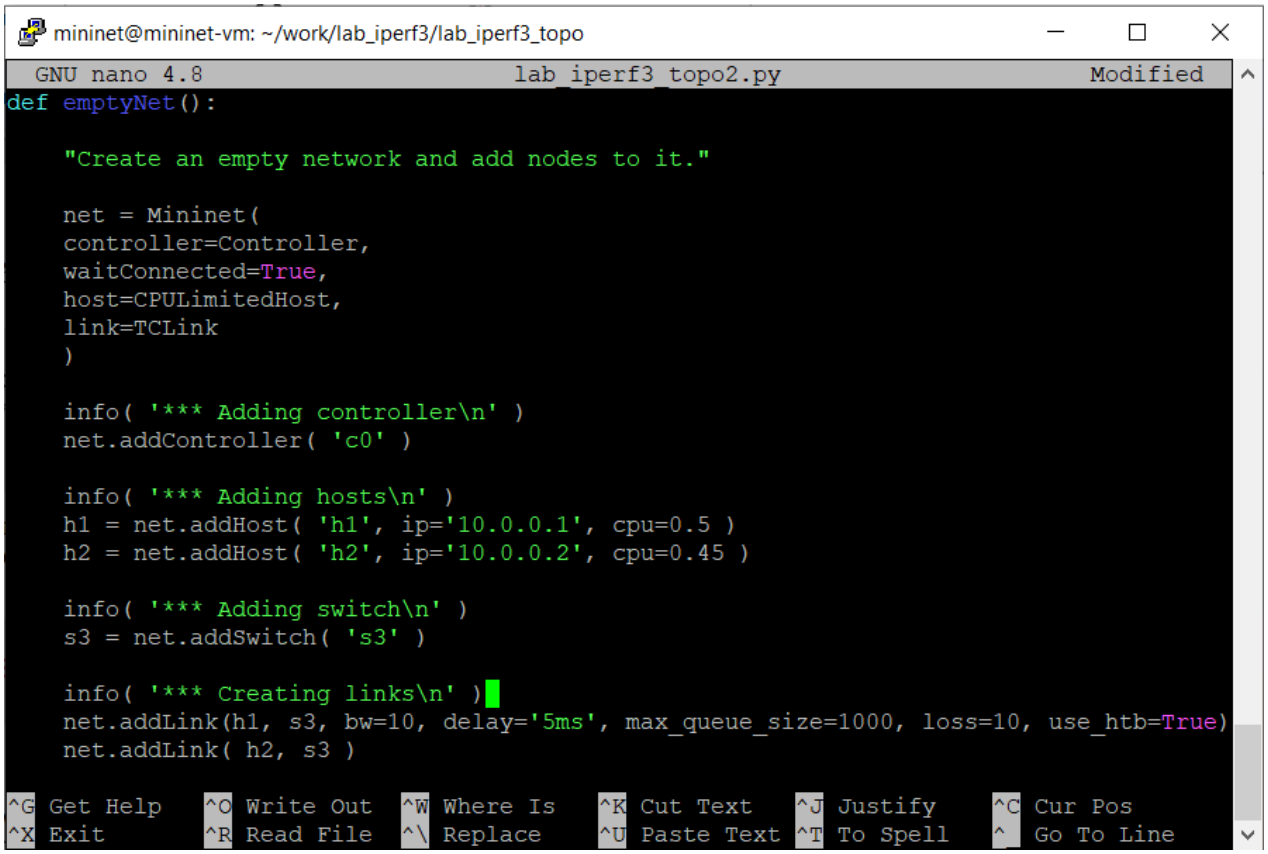


Рис. 12: Настройка скорости, задержки и потерь канала.

При запуске отображены параметры 10 Mbit, 5 ms и 10% потерь; конфигурация CPU применяется через CFS.

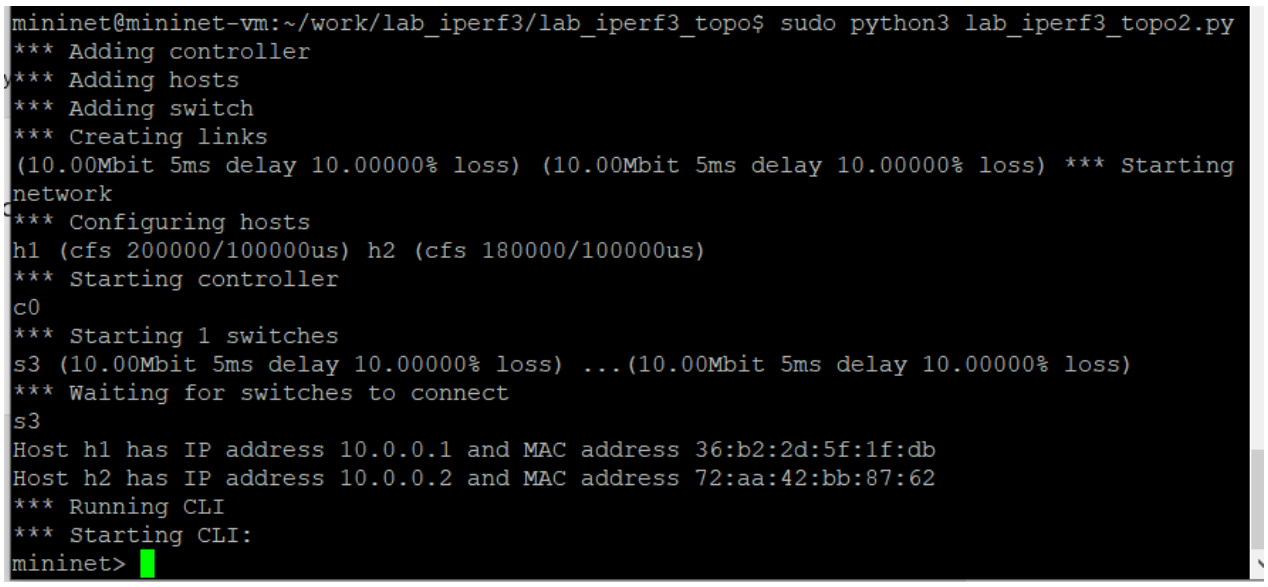


Рис. 13: Запуск сети с заданными ограничениями.

tc qdisc и tc class показывают ограничение до 10 Mbit, задержку 5 ms, 10% потерь и очередь 1000 пакетов.

```
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> h1 tc qdisc show dev h1-eth0
qdisc htb 5: root refcnt 2 r2q 10 default 0x1 direct_packets_stat 0 direct_qlen 1000
qdisc netem 10: parent 5:1 limit 1000 delay 5.0ms loss 10%
mininet> h1 tc class show dev h1-eth0
class htb 5:1 root leaf 10: prio 0 rate 10Mbit ceil 10Mbit burst 15Kb cburst 1600b
mininet>
```

Рис. 14: Проверка HTB и netem через tc.

2.4. Сравнение качества передачи ICMP

Для ограниченной и обычной топологий проведены одинаковые серии по 20 ICMP-запросов. Условия и наблюдения сведены в сравнительную таблицу ниже.

Показатель	Обычная сеть	Сеть с ограничениями
Отправлено / получено	**20 / 20**	**20 / 17**
Потери	**0%**	**15%**
Средний RTT	**0,176 мс**	**187,525 мс**
Минимальный RTT	**0,056 мс**	**10,454 мс**
Максимальный RTT	**1,518 мс**	**1034,513 мс**

Таблица 1 — Сравнение ICMP-измерений для двух топологий.

Из 20 отправленных пакетов получено 17: потери 15%, средний RTT 187,525 мс.

```
mininet> h1 ping -c 20 -i 0.2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=628 ms
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1035 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=167 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=831 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=370 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=11.1 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=15.8 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=20.0 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=23.1 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=11.0 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=11.0 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=11.5 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=11.1 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 17 received, 15% packet loss, time 4003ms
rtt min/avg/max/mdev = 10.454/187.525/1034.513/318.533 ms, pipe 5
mininet>
```

Рис. 15: ICMP-измерения в ограниченной сети.

При 20 запросах получено 20 ответов: потери 0%, средний RTT 0,176 мс.

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python3 lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 46:b5:5c:ba:c5:38
Host h2 has IP address 10.0.0.2 and MAC address ca:50:b0:76:cb:2a
*** Running CLI
*** Starting CLI:
mininet> h1 ping -c 20 -i 0.2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.52 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.559 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.071 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.081 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.071 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.081 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.074 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.070 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.080 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.098 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.118 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.129 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.056 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.076 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=0.111 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=0.065 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 4381ms
rtt min/avg/max/mdev = 0.056/0.176/1.518/0.325 ms
mininet>
```

Рис. 16: ICMP-измерения в сети без ограничений.

При настройке loss=10 вероятность потери оценивается для прохождения пакета по ограниченной линии. Успешный ICMP-обмен требует прохождения запроса и ответа, поэтому 15% потерь за 20 запросов — возможный результат; для оценки вероятности нужны более длинные серии. Минимальное RTT в ограниченной сети согласуется примерно с 5 мс задержки на каждом направлении. Большие пики RTT нельзя уверенно объяснить без дополнительных данных.

2.5. Подготовка воспроизводимого эксперимента

Для автоматизации создан отдельный скрипт: два канала получают одинаковые параметры, а команда запуска iPerf3 формируется программно.

Для отдельного эксперимента создан каталог iperf3, куда перенесён скрипт lab_iperf3.py.

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cp lab_iperf3_topo2.py lab_iperf3
.py
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mkdir -p ~/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mv lab_iperf3.py  ~/work/lab_iper
f3/iperf3/
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cd ~/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ pwd
/home/mininet/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls -lh
total 4.0K
-rwxrwxr-x 1 mininet mininet 1.4K Oct 10 07:28 lab_iperf3.py
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 17: Создание каталога автоматизированного эксперимента.

Подключена библиотека time, из Mininet убран CPULimitedHost; адреса хостов сохранены.

mininet@mininet-vm: ~/work/lab_iperf3/iperf3

```
  GNU nano 4.8                                lab_iperf3.py
import time
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

from mininet.link import TCLink
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet(
    controller=Controller,
    waitConnected=True,
    link=TCLink
    )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1')
    h2 = net.addHost( 'h2', ip='10.0.0.2')


^G Get Help   ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C
^X Exit       ^R Read File  ^\ Replace    ^U Paste Text ^T To Spell   ^
```

Рис. 18: Подготовка скрипта без ограничений CPU.

Для обоих соединений установлены bw=100, delay=75ms, loss=0 и use_htb=False.

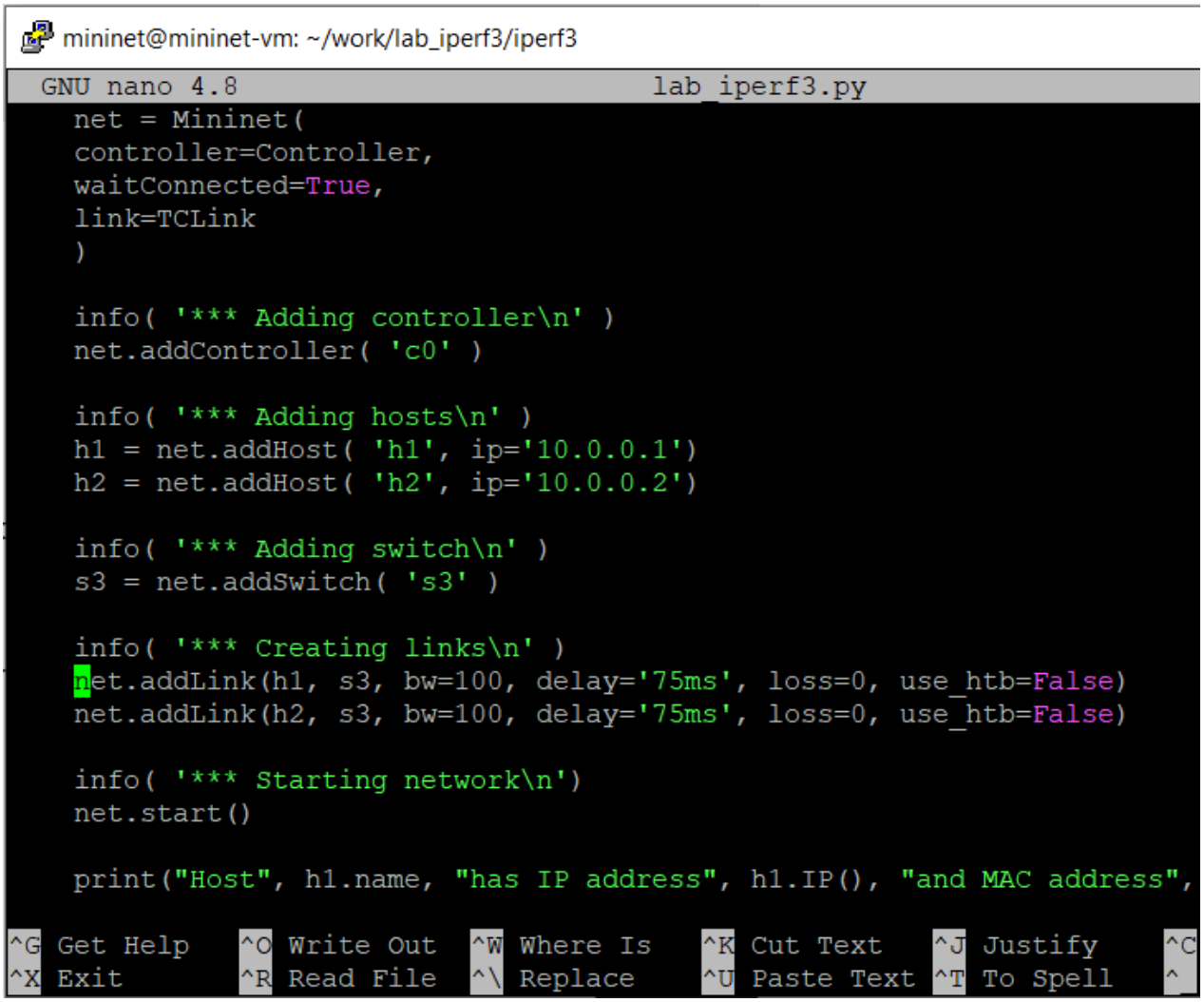


Рис. 19: Настройка обоих каналов на 100 Мбит/с.

Сервер запускается на h2 в режиме демона, через 10 секунд клиент h1 выполняет тест с ключом -J и сохраняет JSON.

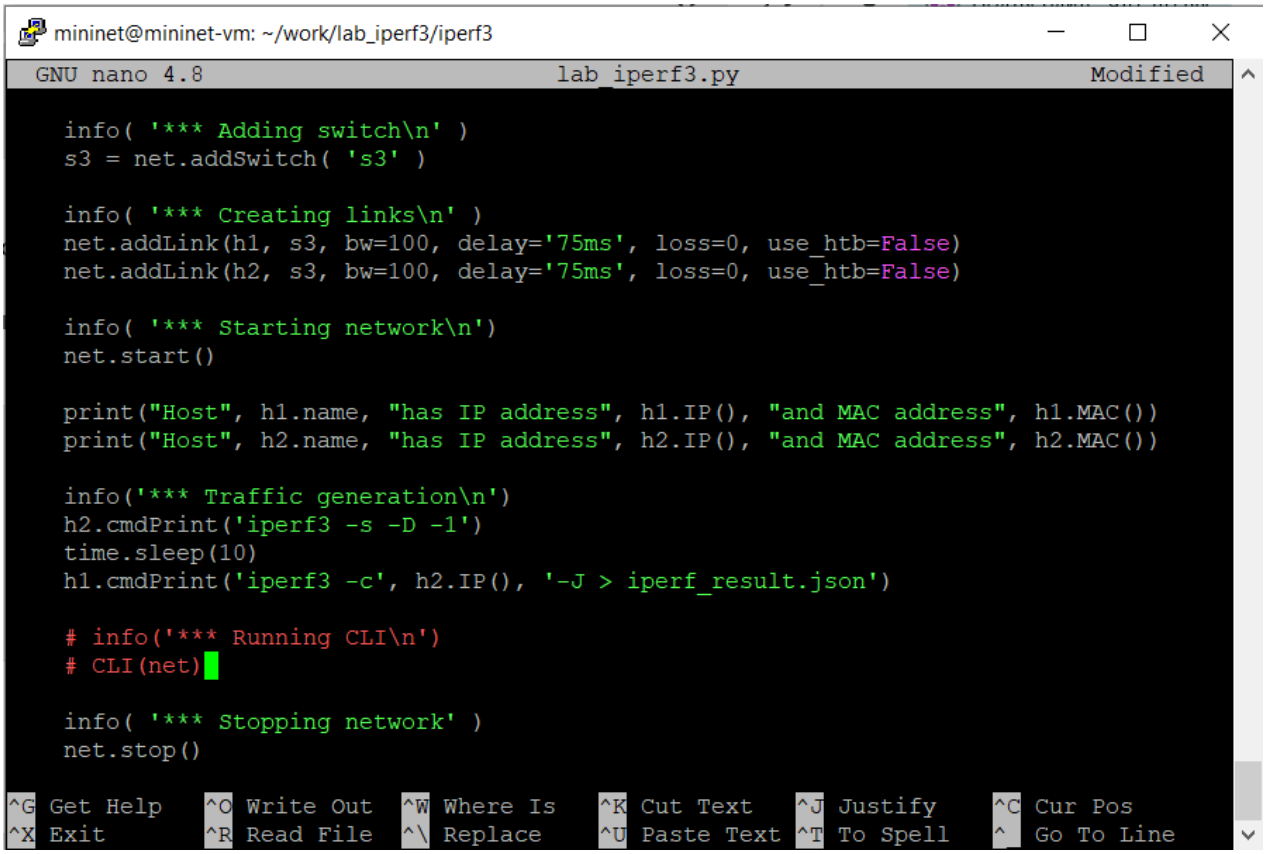


Рис. 20: Программный запуск iPerf3 и сохранение JSON.

2.6. Автоматический запуск и обработка JSON

После запуска серверной части клиент выполняет десятисекундный TCP-тест. JSON проверяется синтаксически и содержит численное значение измеренной скорости.

Скрипт создал сеть, вызвал сервер и клиент iPerf3, затем завершил работу Mininet без входа в CLI.

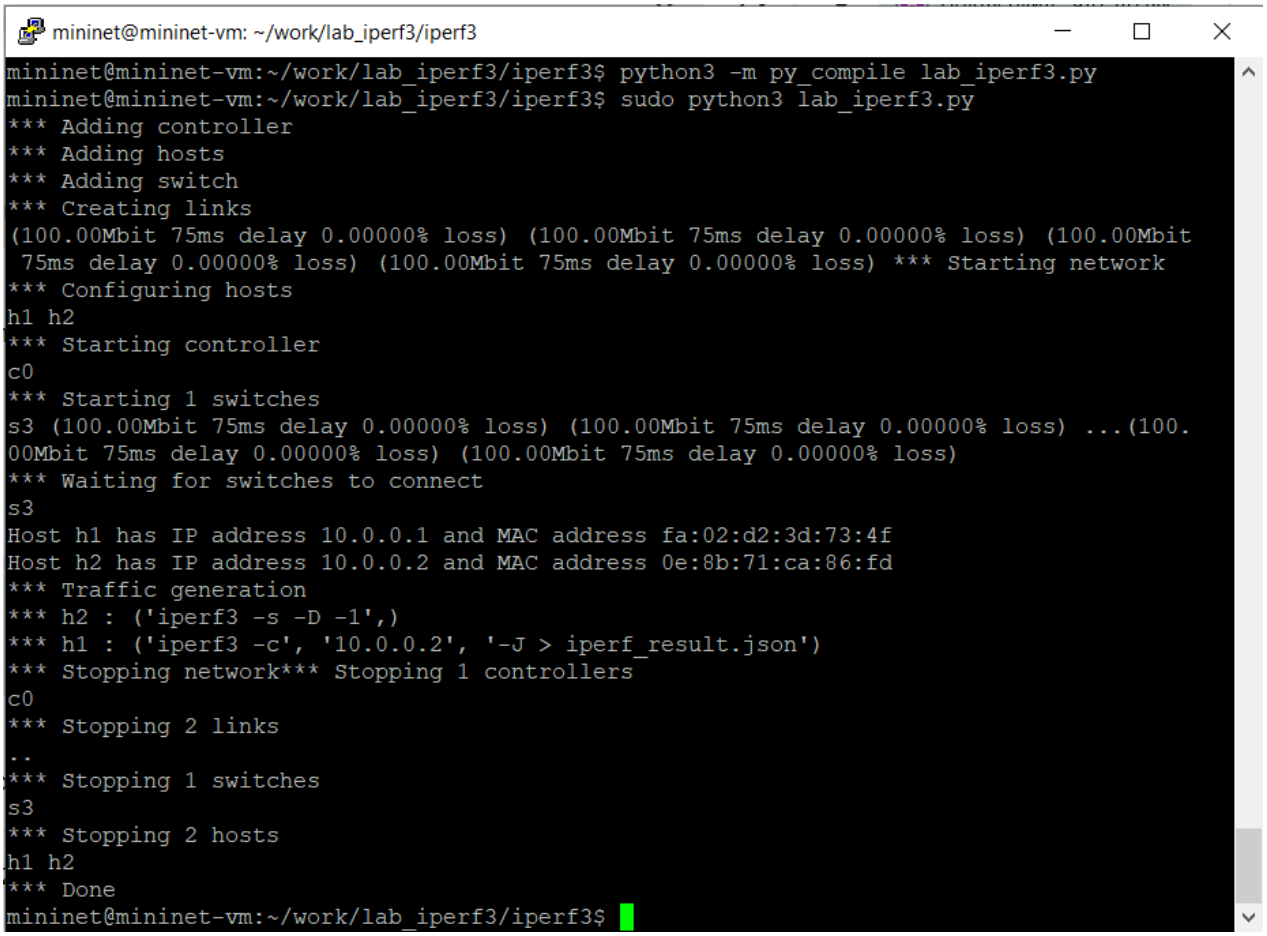


Рис. 21: Выполнение автоматизированного эксперимента.

Файл iperf_result.json прошёл проверку JSON VALID; ошибок iPerf3 нет, скорость получена около 9,97 Мбит/с.

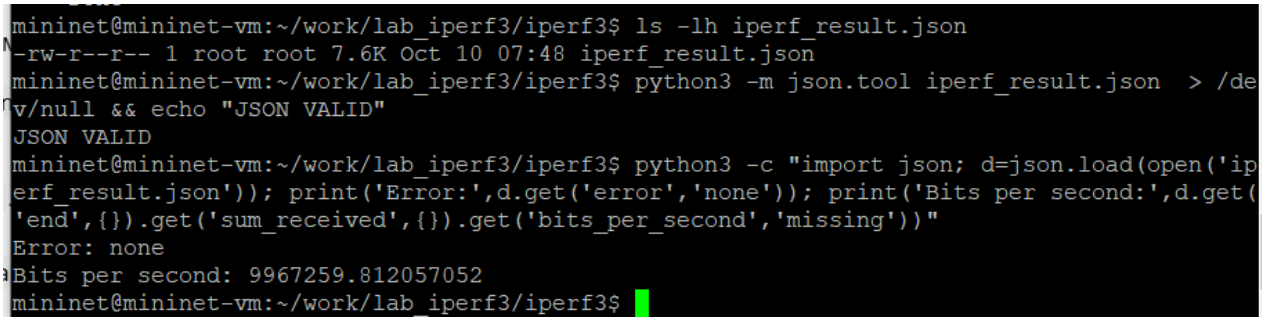


Рис. 22: Проверка первого JSON-результата.

Скрипт plot_iperf.sh построил семь PDF-графиков и вспомогательный набор данных 1.dat.

```
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ sudo chown mininet:mininet iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ plot_iperf.sh iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls -lh results/
total 88K
-rw-rw-r-- 1 mininet mininet  528 Oct 10 07:54 1.dat
-rw-rw-r-- 1 mininet mininet 9.5K Oct 10 07:54 bytes.pdf
-rw-rw-r-- 1 mininet mininet 9.5K Oct 10 07:54 cwnd.pdf
-rw-rw-r-- 1 mininet mininet 8.9K Oct 10 07:54 MTU.pdf
-rw-rw-r-- 1 mininet mininet 8.8K Oct 10 07:54 retransmits.pdf
-rw-rw-r-- 1 mininet mininet 8.8K Oct 10 07:54 RTT.pdf
-rw-rw-r-- 1 mininet mininet 9.1K Oct 10 07:54 RTT_Var.pdf
-rw-rw-r-- 1 mininet mininet 9.3K Oct 10 07:54 throughput.pdf
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 23: Генерация PDF-графиков.

2.7. Визуализация результатов

Графики позволяют визуально сопоставить изменение пропускной способности, окна перегрузки TCP и времени получения подтверждений.

Кривая throughput показывает изменение скорости в течение 10-секундного TCP-теста; скорость заметно меняется со временем.

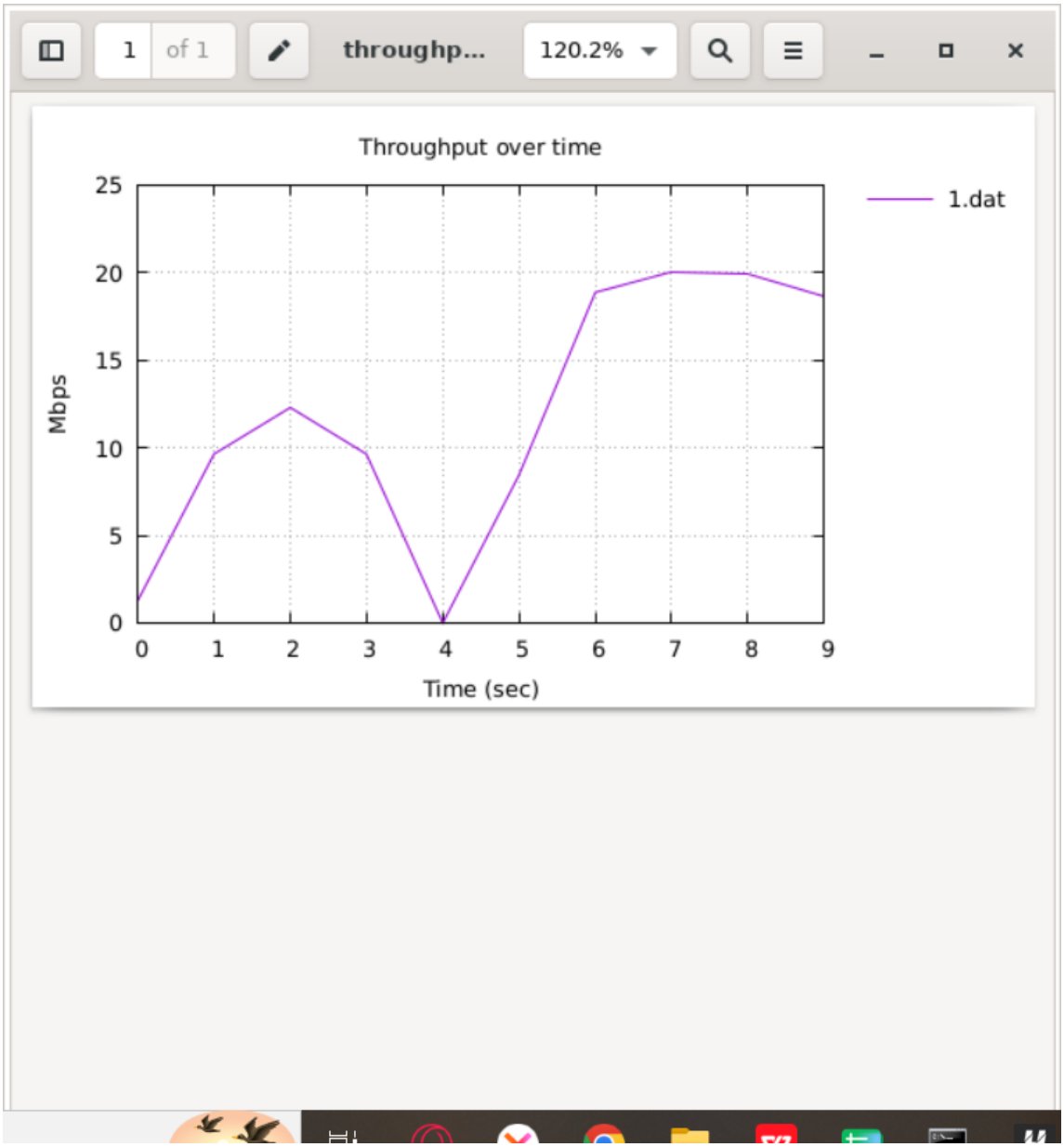


Рис. 24: График пропускной способности.

На графике cwnd виден рост значения окна перегрузки TCP в течение теста.

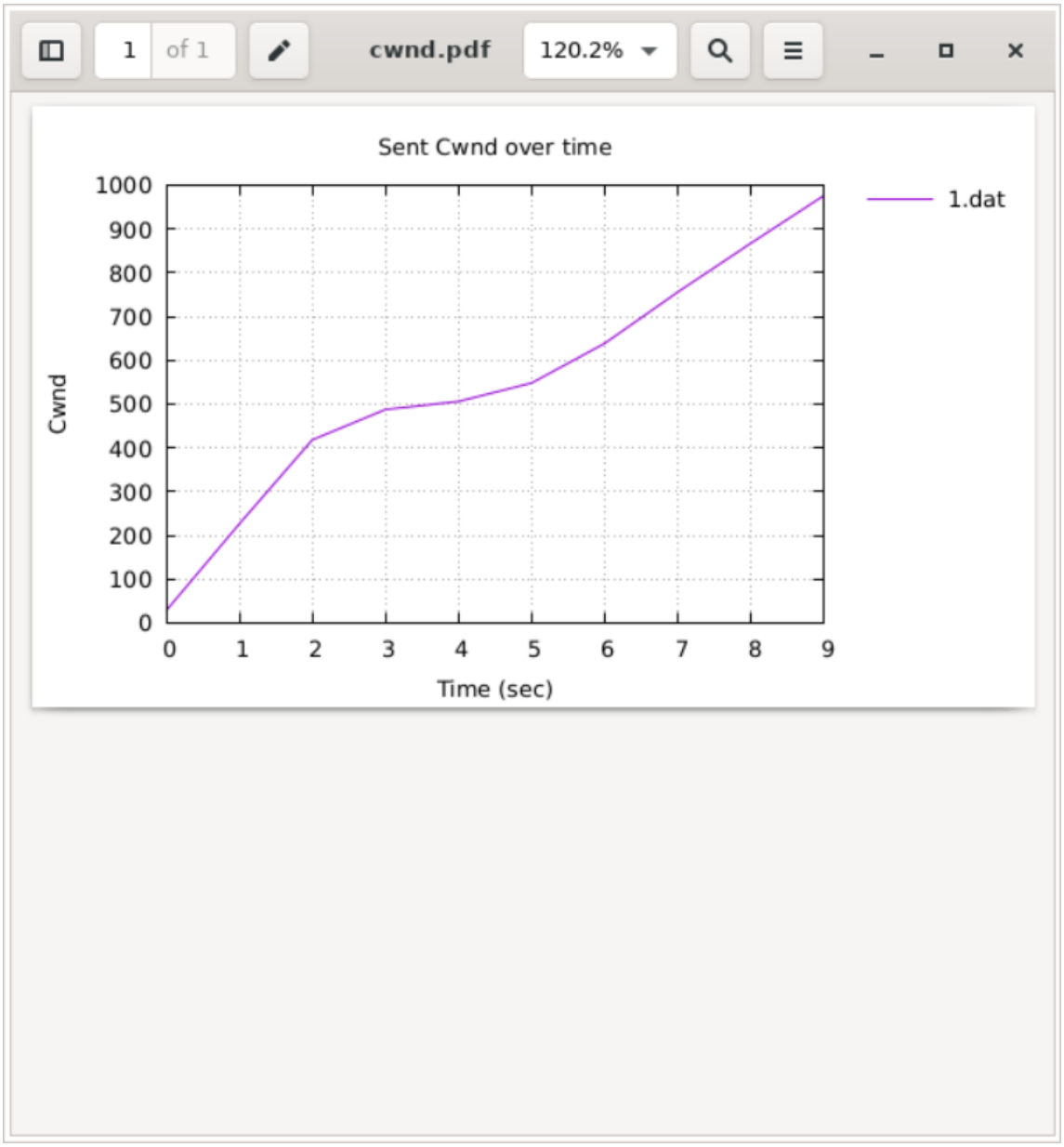


Рис. 25: График окна перегрузки TCP.

Кривая RTT отражает колебания времени подтверждения; исходная числовая шкала графика требует проверки единиц измерения.

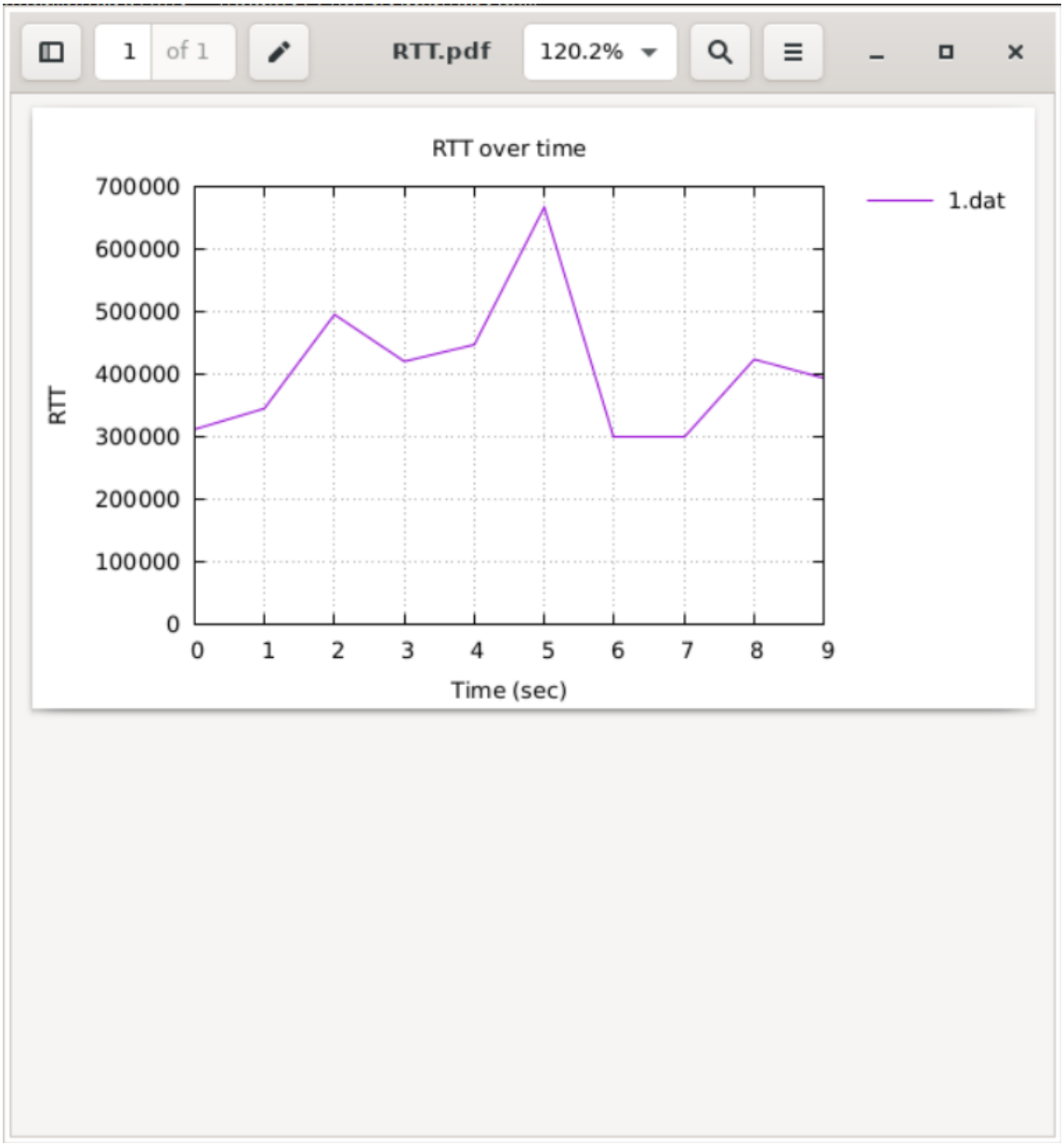


Рис. 26: График RTT.

Визуально график throughput отражает неравномерную скорость, а cwnd — рост окна перегрузки в ходе передачи. График RTT показывает изменения времени отклика; числовую шкалу нельзя без проверки единиц измерения трактовать как миллисекунды.

2.8. GNU Make и проверка воспроизводимости

После создания Makefile достаточно выполнить make clean и make. При повторном запуске скрипт заново сформировал результаты и графики без ручного запуска сети.

Makefile содержит цели all, iperf_result.json, plot и clean для повторяемой последовательности действий.

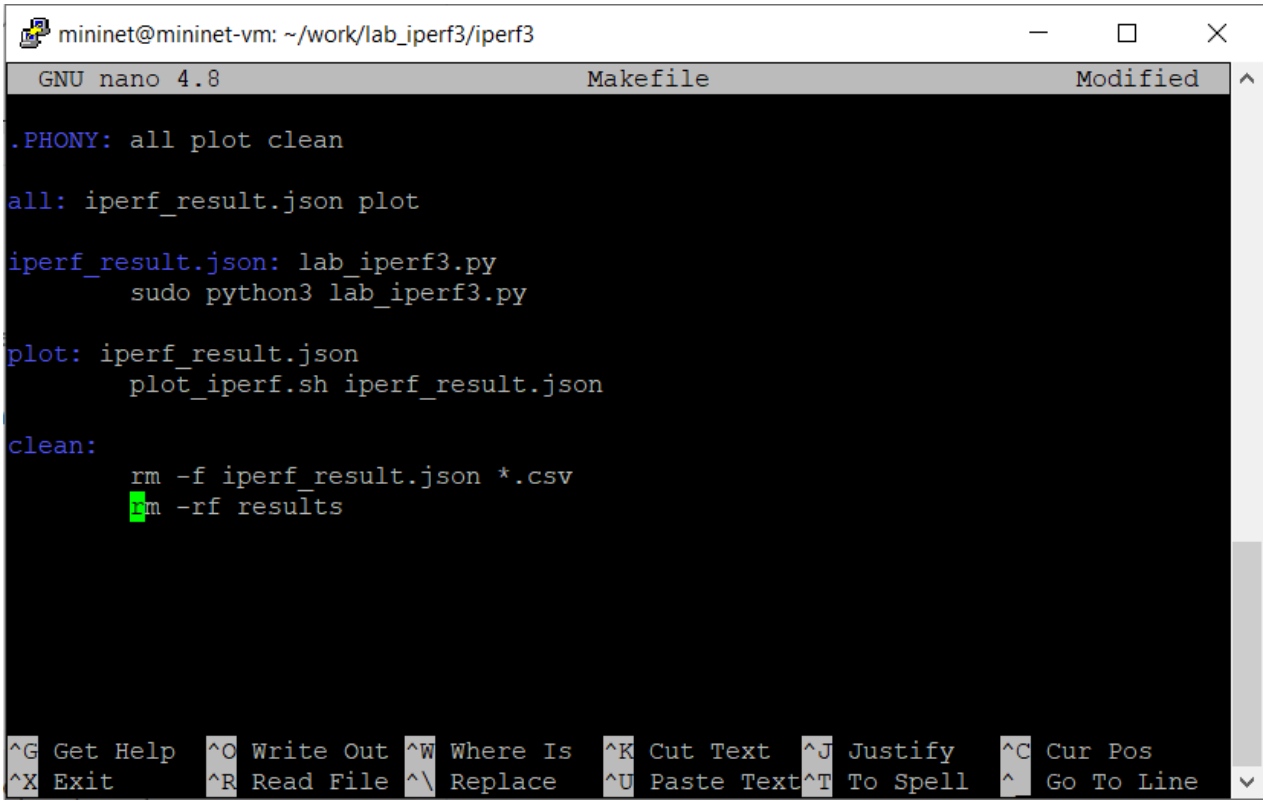


Рис. 27: Правила автоматизации в Makefile.

Команды make clean и make повторно создали сеть, вызвали iPerf3 и построили графики.

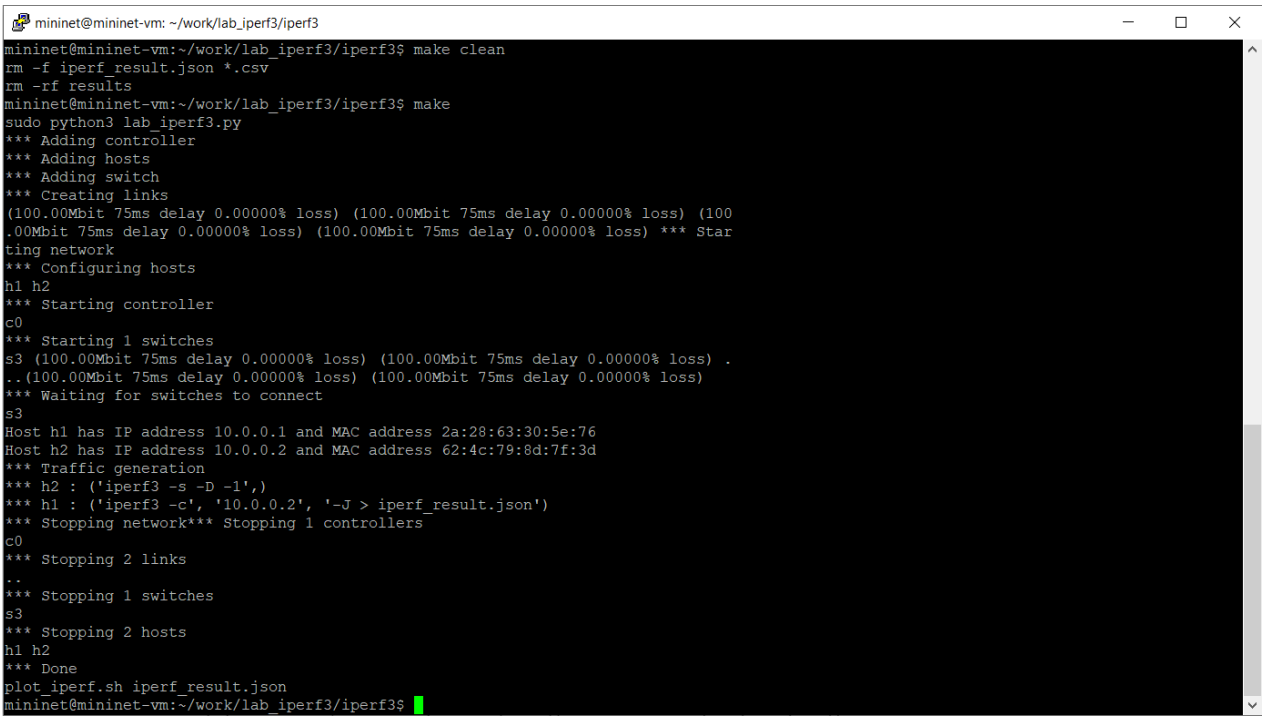


Рис. 28: Повторный запуск после очистки результатов.

После make снова созданы JSON и PDF; JSON VALID, Error: none, пропускная способность 35,62 Мбит/с.

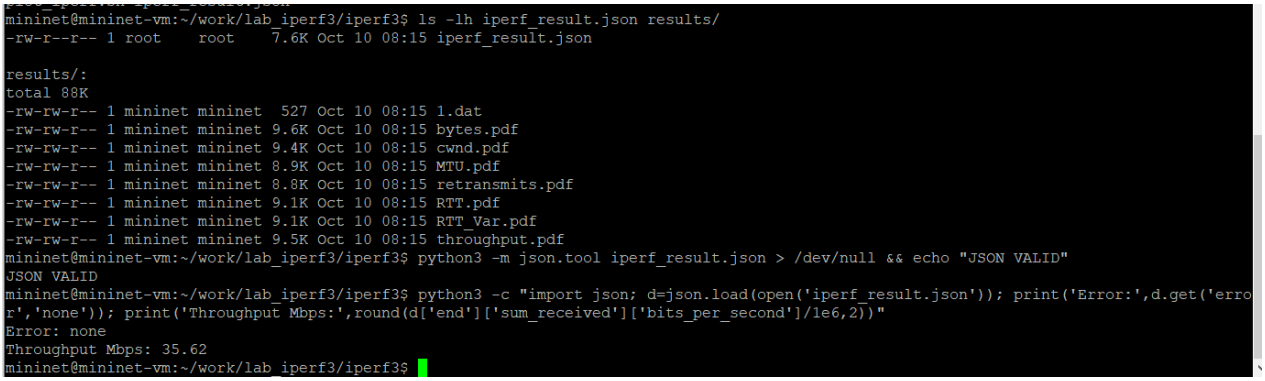


Рис. 29: Проверка итоговых файлов и результата.

3. Полученные результаты и анализ

Исходная топология: связи h1–s3 и h2–s3 работали, pingall завершился без потерь.
Ограниченная топология: настройки HTB/netem подтверждены tc; в коротком ICMP-тесте потери составили 15%, средний RTT — 187,525 мс, против 0% и 0,176 мс в сети без ограничений.

Показатель	Первый автоматический запуск	Повторный запуск после make clean
Заданная скорость каждого канала	**100 Мбит/с**	**100 Мбит/с**
Заданная задержка каждого канала	**75 мс**	**75 мс**
Полученная TCP-скорость	**9,97 Мбит/с**	**35,62 Мбит/с**
Корректность JSON	**JSON VALID**	**JSON VALID**
Ошибка iPerf3	**отсутствует**	**отсутствует**

Таблица 2 — Сопоставление автоматических TCP-экспериментов.

Разница между измеренными скоростями составила 25,65 Мбит/с; во втором запуске скорость была приблизительно в 3,57 раза выше. Это подтверждает **воспроизводимость процедуры** (топология создаётся, тест запускается, JSON и графики формируются), но **не подтверждает стабильность численного результата**. Возможные причины различий — работа алгоритма TCP, его окно перегрузки, задержки, буферизация и состояние виртуальной машины; по двум измерениям выбрать одну доказанную причину нельзя.

Для статистической проверки повторяемости потребовалась бы серия независимых запусков с фиксированными условиями и сравнением скоростей, RTT, повторных передач и нагрузки на CPU. Дополнительная серия в рамках данной работы не выполнялась.

4. Заключение

Выполнены построение топологии Mininet через Python API, настройка хостов и каналов, проверка IP/MAC-адресов, ICMP-измерения и проверка HTB/netem. Автоматический эксперимент iPerf3 создал корректный JSON и семь PDF-графиков. Команды make clean и make повторно запустили полный цикл без ручного управления Mininet. При этом зафиксировано заметное различие TCP-скорости между двумя прогонами — важное ограничение численной повторяемости.

5. Библиографическая информация

1. Лабораторная работа № 3 «Измерение и тестирование пропускной способности сети. Воспроизводимый эксперимент». Учебно-методические материалы по дисциплине «Моделирование сетей передачи данных».

2. Mininet. Documentation and examples. https://mininet.org/

3. iPerf3. Documentation. https://software.es.net/iperf/

4. GNU Make Manual. https://www.gnu.org/software/make/manual/

5. ekfoury/iperf3_plotter. https://github.com/ekfoury/iperf3_plotter

6. Перечень иллюстраций и таблиц

- Рис. 1: Подготовка рабочей папки и копирование emptynet.py.
- Рис. 2: Исходный скрипт создания сети.
- Рис. 3: Запуск исходной топологии через Python API.
- Рис. 4: Проверка связности и состава сети.
- Рис. 5: Добавление вывода параметров хоста h1.
- Рис. 6: Вывод IP- и MAC-адреса h1.
- Рис. 7: Добавление вывода параметров второго хоста.
- Рис. 8: Результаты получения адресов обоих хостов.
- Рис. 9: Подключение классов CPULimitedHost и TCLink.
- Рис. 10: Настройка классов узлов и соединений Mininet.
- Рис. 11: Назначение ограничений CPU хостам.
- Рис. 12: Настройка скорости, задержки и потерь канала.
- Рис. 13: Запуск сети с заданными ограничениями.
- Рис. 14: Проверка HTB и netem через tc.
- Рис. 15: ICMP-измерения в ограниченной сети.
- Рис. 16: ICMP-измерения в сети без ограничений.
- Рис. 17: Создание каталога автоматизированного эксперимента.
- Рис. 18: Подготовка скрипта без ограничений CPU.
- Рис. 19: Настройка обоих каналов на 100 Мбит/с.
- Рис. 20: Программный запуск iPerf3 и сохранение JSON.
- Рис. 21: Выполнение автоматизированного эксперимента.
- Рис. 22: Проверка первого JSON-результата.
- Рис. 23: Генерация PDF-графиков.
- Рис. 24: График пропускной способности.
- Рис. 25: График окна перегрузки TCP.
- Рис. 26: График RTT.
- Рис. 27: Правила автоматизации в Makefile.
- Рис. 28: Повторный запуск после очистки результатов.
- Рис. 29: Проверка итоговых файлов и результата.
- Таблица 1 — Сравнение ICMP-измерений для двух топологий.
- Таблица 2 — Сопоставление автоматических TCP-экспериментов.